Watsonx Orchestrate Lab Guide: Create a holiday agent using remote MCP servers & assemble a multiagent AI system

The Art of IBM's AI Customization:

Your Enterprise-Ready multiagents AI system

Authors: Joost Vos, joost.vos@ibm.com

Marc Decker, marc.decker@nl.ibm.com

Andrea Longo, andrea.longo@ibm.com

Version: 1.0 / April 14th, 2026

Image: Herbert Goetsch on https://unsplash.com/photos/brown-framed-sunglasses-VMszlfU17ds

1. IBM WATSONX ORCHESTRATE — INTRODUCTION FOR A HANDS-ON LAB 3

1.1.1 What is watsonx Orchestrate? 3

1.2. THE KEY BUILDING BLOCKS YOU'LL USE IN A LAB 4

1.3 CALLING TOOLS THROUGH MODEL CONTEXT PROTOCOL (MCP) 5

1.3.1 Why does MCP make your life easier? 5

2 WHAT YOU WILL LEARN IN THIS LAB 6

2.1 Requirements 6

3.1 PART 1 - BUILDING YOUR MCP-BASED HOLIDAY AGENT 8

3.2 PART 2 - ADDING A REMOTE MCP SERVER WITH AUTHENTICATION 18

4 MULTIAGENT SYSTEMS AND IBM WATSONX ORCHESTRATE 24

4.1 The Agent2Agent (A2A) protocol 24

4.1.1 Why Agent2Agent and MCP matters? 25

4.1.2 The IBM watsonx Orchestrate's Open Framework Architecture 25

4.2 Let's play with A2A and create your first multiagent system. 26

4.2.1 The basics: How watsonx Orchestrate handles intent and routing 26

4.2.2 Why this is interesting? 27

4.2.3 The bigger A2A (and MCP) picture 27

5 PART 3 – BUILD YOUR FIRST MULTIAGENT SYSTEM 28

1. IBM watsonx Orchestrate — introduction to the labs

IBM watsonx Orchestrate is an **AI-driven automation and orchestration platform** that helps you turn **work into repeatable, assistable flows**. In simple terms: it lets you build an **AI (multi)agents** and **AI assistants** that can **understand a task request, decide what needs to happen**, and **carry out steps across business tools and systems**—with appropriate guardrails and human oversight.

For this hands-on lab, the most useful mental model is:

- **You describe the work:** what outcome you want, plus the rules
- **You connect the tools:** systems it can read/write, like ticketing, email, HR, CRM, internal APIs.
- **You make the AI reliable:** permissions, approvals, policies, and testing.
- **You run it like a product:** monitoring, iteration, and change control.

1.1.1 What is watsonx Orchestrate?

In C-level management terms, think of watsonx Orchestrate as a **digital coworker augmenting employee productivity** in completing repetitive business tasks:

- It can **guide** a user through a request or task ("what do you need?").
- It can **automate and complete** (parts of the) work ("I created the ticket, notified the team, and booked the meeting").
- It can **handoff** to a human when needed ("I need approval before I reset access").

Technically, watsonx Orchestrate is an orchestration layer where an AI agents and AI assistants coordinate:

- **Natural-language interaction:** chat-style requests
- **Tool use:** connectors/integrations, APIs, automations
- **Task/skill execution:** structured actions with inputs/outputs
- **Governance:** authentication, authorization, auditability, approvals
- **Operationalization:** monitoring, refinement, lifecycle

This combination is what people mean by **agentic AI** in practice: not just generating text, but **planning and acting** using tools.

1.2. The key building blocks you'll use in a lab

Although IBM uses specific product terms in the watsonx Orchestrate user interface, most labs boil down to these concepts:

1. The "Front Door": AI Agent/Assistant

The user interacts with the agent/assistant via chat or embedded UI. The chat interface:

- collects intent and context
- asks follow-up questions when inputs are missing
- triggers actions and tracks progress

2. The "Hands": Skills / Actions / Tasks / Tool calling

The "hands" – and depending on the platform they carry different names, like skills, actions - represent **repeatable capabilities** like:

- Create an incident
- Look up a customer
- Reset a password
- Summarize last week's changes and draft an email

3. The "Tools": Integrations / connectors

AI orchestration becomes extremely valuable when it can interact with real (enterprise) systems, through API integrations and connections.

- SaaS apps (tickets, docs, chat, CRM)
- enterprise systems
- custom endpoints (your own APIs)

To increase flexibility, modularity, security and governance, Model Context Protocol (MCP) servers can be leveraged for maintaining APIs that can be consumed in AI applications, like AI agents. In a lab, you'll configure an AI agent that calls a tool through a remote Model Context Protocol Server (MCP server) to complete one task end-to-end:

User input → action → Tool calling → outputs → user message.

Read more about MCP servers on the next page.

4. The “seatbelts”: Guardrails and governance

Agentic systems are designed to pursue goals, decide, and act over time within defined boundaries. Typical guardrails include:

- role-based access
- step-level approvals (“ask before ....”)
- logging/auditing
- data handling rules (what can be sent to which tool)

1.3 Calling tools through Model Context Protocol (MCP)

Model Context Protocol (MCP) is an open standard that defines how AI applications talk to external tools and data sources. Think of it as a universal plug. Just like USB gave us a universal connector for keyboards, printers, and storage drives, MCP gives AI assistants one universal and consistent way to discover, understand, and use tools—whether those tools talk to a database, a SaaS application, an internal service, or a file system. To read more on the (technical) background of MCP, navigate to https://modelcontextprotocol.io/docs/getting-started/intro.

1.3.1 Why does MCP make your life easier?

As AI assistants become more capable, they need to do more than generate text—they need to look things up, trigger actions, and work with real systems. Without a standard, every AI-to-tool connection becomes a one-off integration: custom code, custom auth handling, custom error logic, repeated for every new tool and every new assistant. That approach works for a demo, but it doesn't scale. In short: individual API integrations tie a specific agent to a specific system with custom glue code. MCP solves that problem. MCP allows you to **build tools once, use them everywhere in any MCP-compatible AI application, and govern them centrally**. MCP is much more developer friendly than individual API integrations. Adding a new capability to an agent becomes a matter of deploying a new MCP server — no agent changes needed. Authentication and access control can be managed consistently at the protocol level. Structured, uniform tool calls make logging and auditing straightforward. And any MCP-compatible assistant can use the same tool servers out of the box. That's why MCP is one of the preferred approach for connecting AI assistants—including IBM watsonx Orchestrate—to the outside world.

2 What you will learn in this lab

In this lab, you will be learning how to **build a holiday agent using a remote MCP server**. This part of the lab guide describes the steps needed to achieve a functional prototype of a remote MCP server-based holiday agent that allows user to lookup official holidays in >100 countries worldwide.

2.1 Requirements

A. Note: To successfully complete part 3 of this lab, make sure you completed the *wxO L4 - Creating agents with flow and document processing - Lab Guide - V251022.PDF* (Download instructions and assets through the Git repo below)
B. All document data can be found in the following git-repository:

   https://github.com/joost-vos/wxo-with-remote-mcp-handson

C. A **text editor (notepad++, BBEdit)** for collecting URLs. API keys, Project IDs
D. Access to IBM watsonx Orchestrate

For this lab, an IBM Cloud environment will be at your disposal with the required IBM Cloud services. Details on accessing this environment is distributed through e-mail.

2.2 Prep steps

You will configure and integrate the following IBM Cloud component:

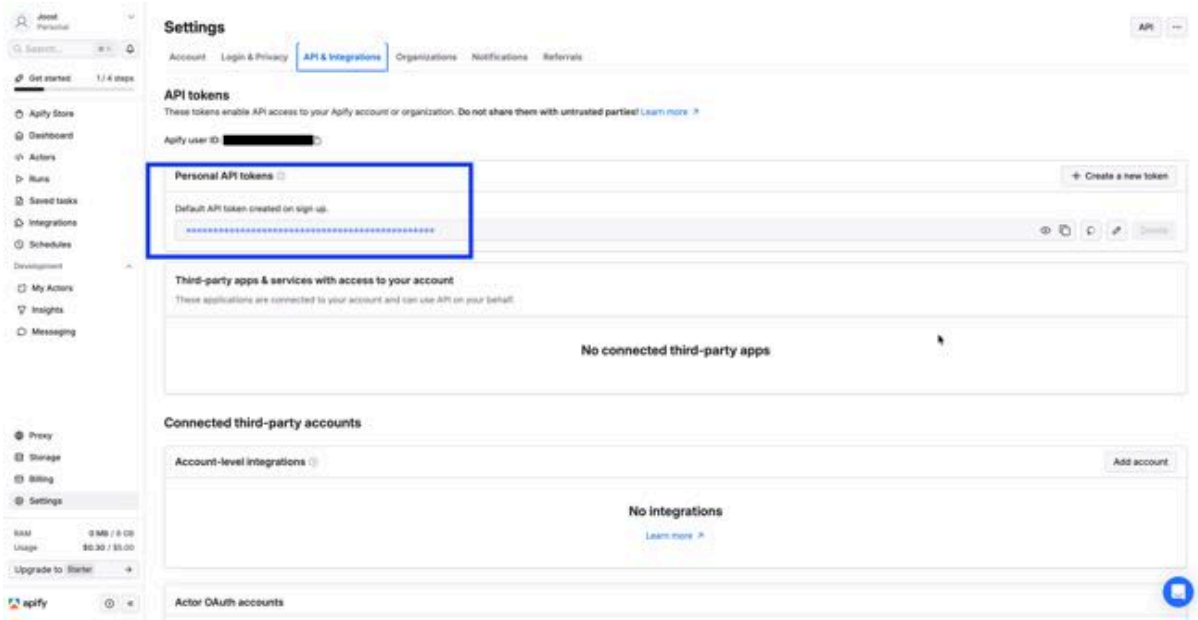
1. **watsonx Orchestrate** – IBM's agentic AI platform

Through watsonx Orchestrate you will call tools through remote MCP servers, one of which without authentication and one with authentication. You will be connecting two remote MCP servers:

1. **Pipeworx.io Holiday MCP server** (Public, no authentication, easy to use)
   See for more info: https://pipeworx.io/
   Address MCP server:
   https://gateway.pipeworx.io/holidays/mcp
2. **Apify Holiday MCP server** (Free tier, with authentication)
   See for more info: https://apify.com/
   Address MCP server:
   https://mcp.apify.com/?tools=call-actor,easyapi/public-holidays-api

Getting your **Apify** Credentials

1. First create an account at apify.com (https://apify.com/)
2. Validate your e-mail address
3. Navigate to the Apify console and navigate to **"settings"** and then to the tab **"API & Integrations", or click the following link:** (https://console.apify.com/settings/integrations)

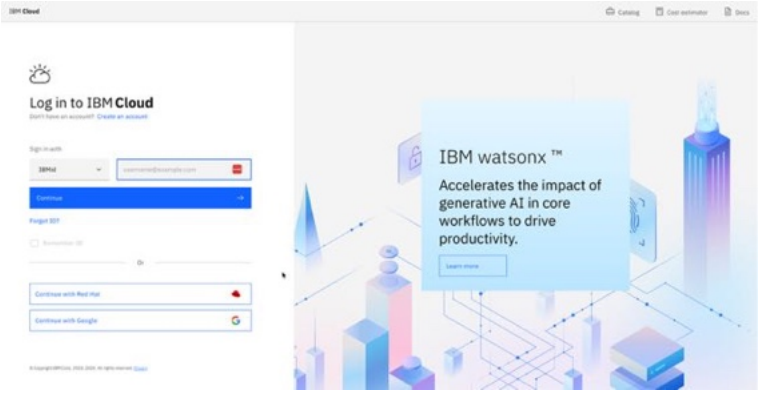


4. Copy & paste the default API token onto a new file in Notepad ++ or BBEdit
5. Copy and paste the following MCP server address to Notepad ++ or BBEdit https://mcp.apify.com/?tools=call-actor,easyapi/public-holidays-api
6. You will be using these later

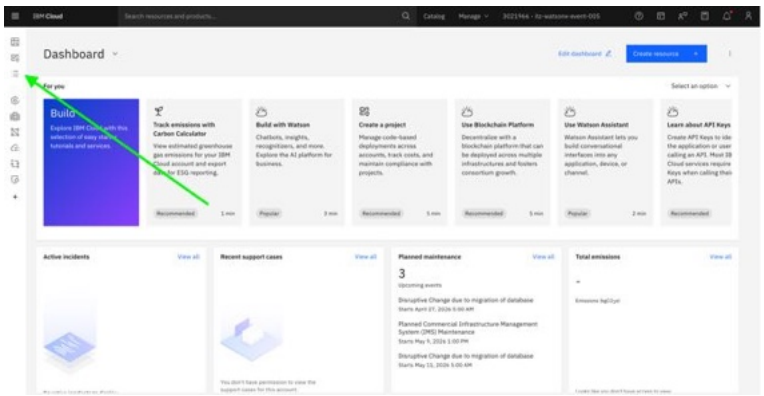
3.1 Part 1 - Building your MCP-based Holiday Agent

Launch IBM watsonx orchestrate

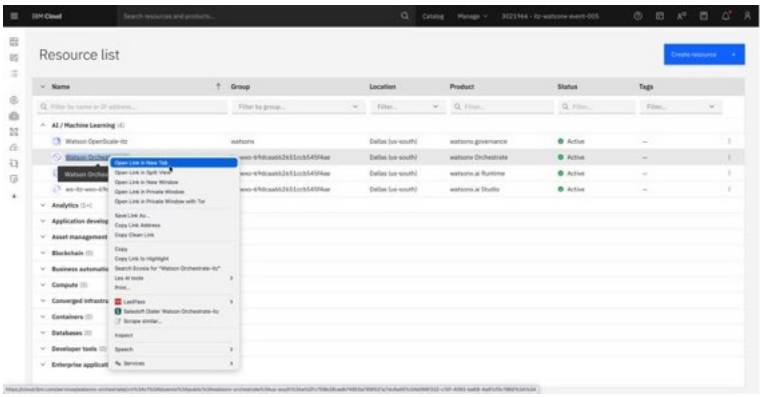
1. Log in onto the IBM Cloud.



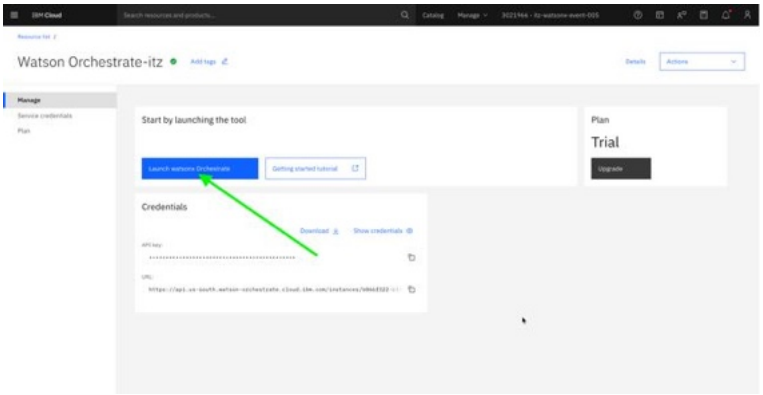
2. Go to the **resources menu** on the left-hand side, indicated by the green arrow.



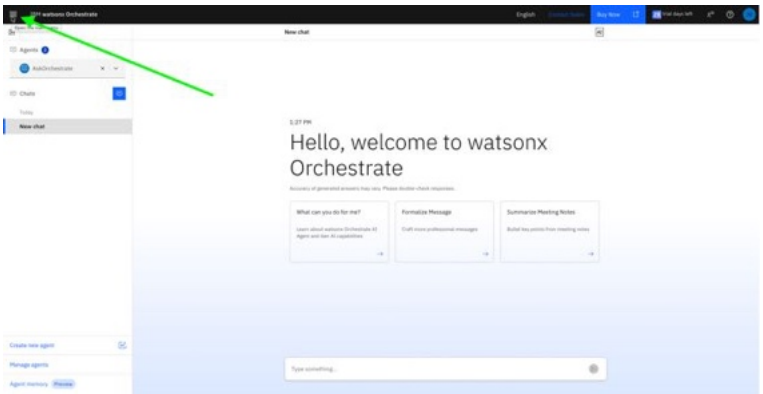
3. Be sure to select the appropriate Cloud Account you have been provided with in the upper right corner in the black top bar. The Account can be recognized by 7 digits, dash itz-watsonx-[0-9] or ask you instructor.
4. Click on the **Watson Orchestrate service**, and open using right click **"Open link in new tab"**



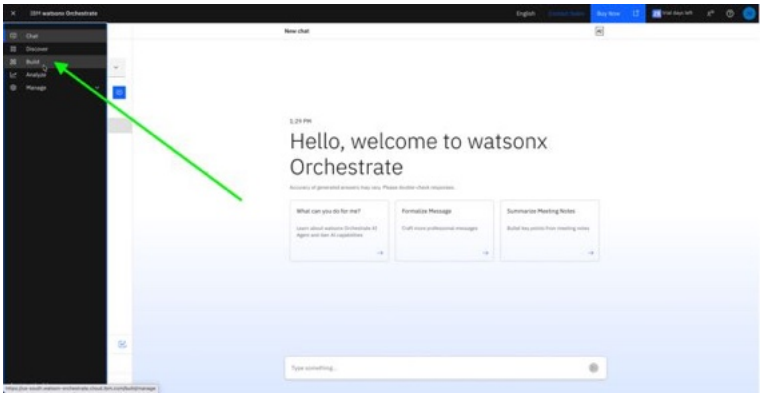
5. Click on the **Launch watsonx Orchestrate** button



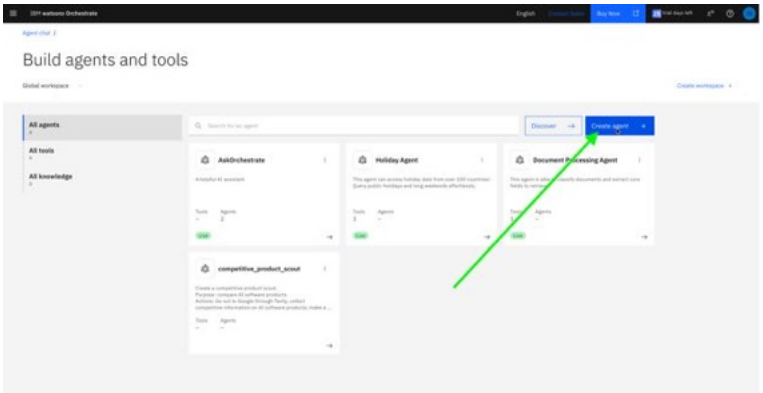
6. Open the **main menu**



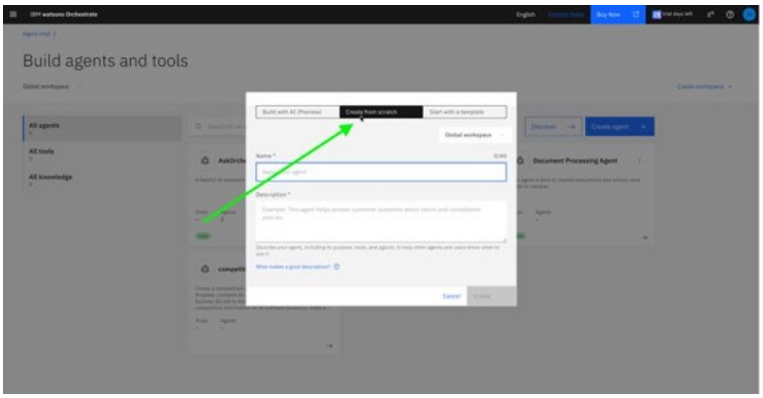
7. Next, click on **Build**



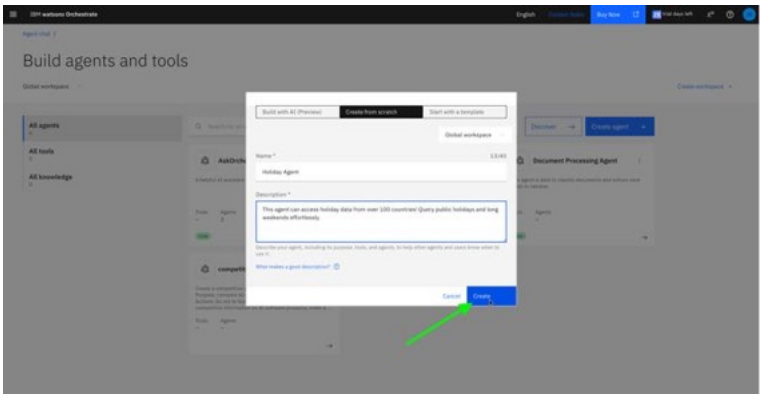
8. Click on the blue “**create agent**” button



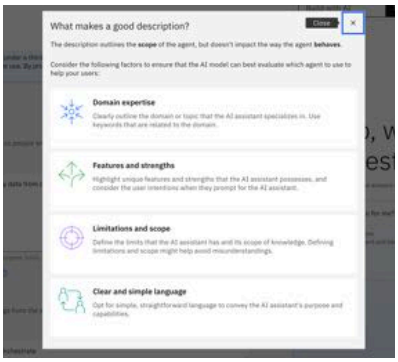
9. Start creating your agent by clicking on “**create from scratch**”



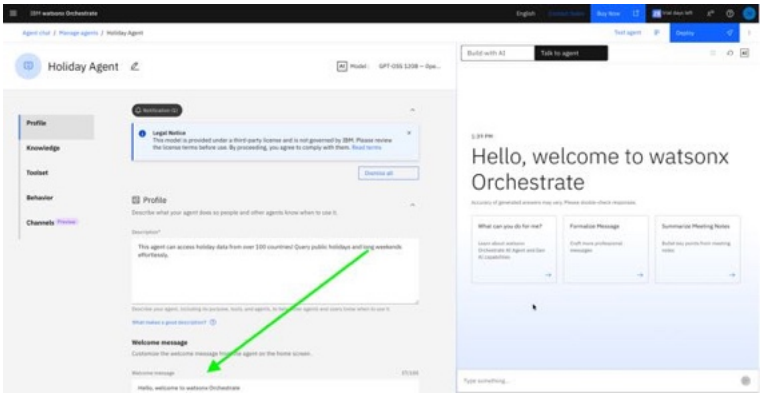
10. Give your Holiday agent the following:
    a. **name:** *Holiday agent*
    b. **description:** *“This agent can access holiday data from over 100 countries! Query public holidays and long weekends effortlessly.”*
    c. Click **Create**



In the next screen, you will be configuring your holiday agent. You will define the 5 key ingredients of an AI agent, namely its **Profile, Knowledge, Toolset, Behavior** and **Channels**. Be sure to give an accurate description of your agent. Adding a proper description, including its purpose, tools, and agents helps other agents and users know how and when to use it. Proper configuration of these ingredients determines the overall effectiveness and accuracy of the agent and muti-agent collaboration.



11. Scroll down to “welcome message”

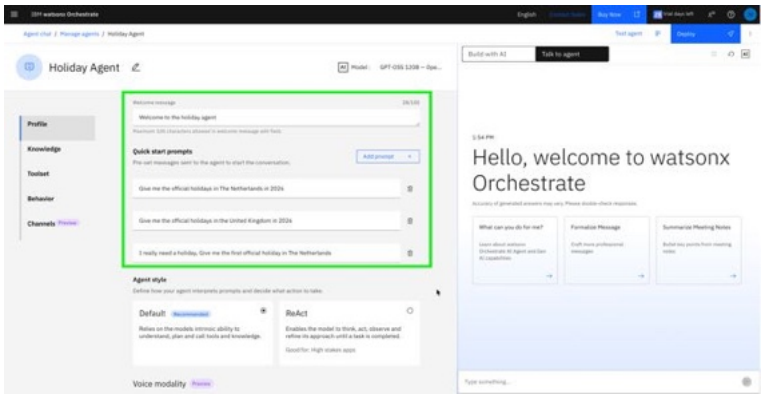


12. Add the following information:

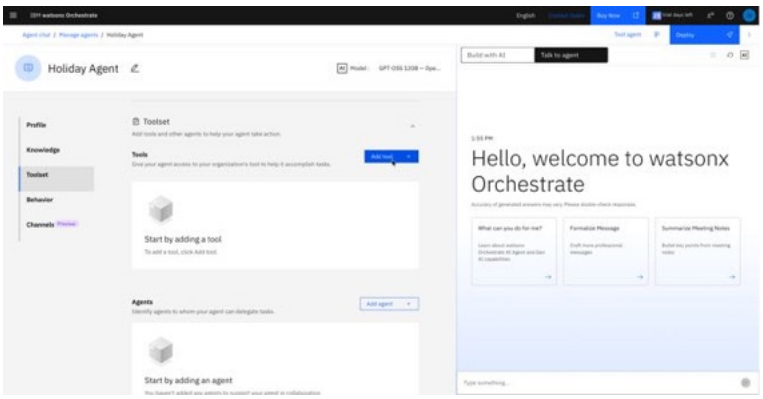
welcome message: *Welcome to the holiday agent*

Quick start prompts:

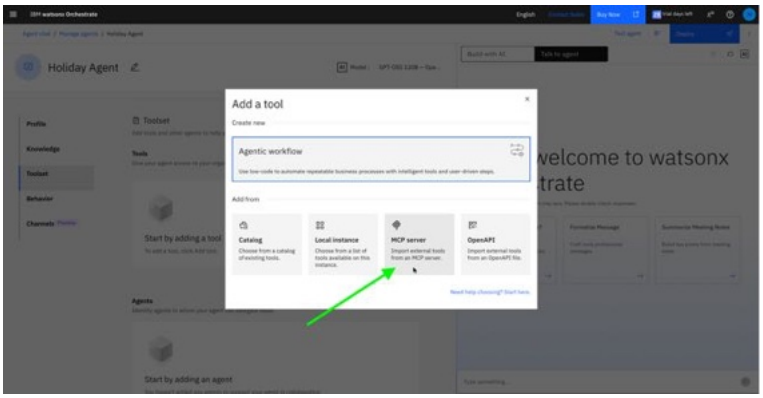
1. *Give me the official holidays in The Netherlands in 2026*
2. Give me the official holidays in the United Kingdom in 2026
3. *I really need a holiday, Give me the first official holiday in The Netherlands*



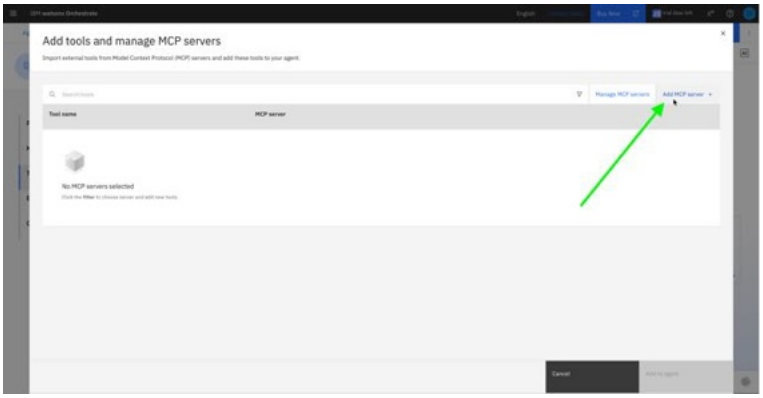
13. Next, scroll to **Toolsets** (or click on **Toolset** in the left menu) and click on the blue **Add tool** button



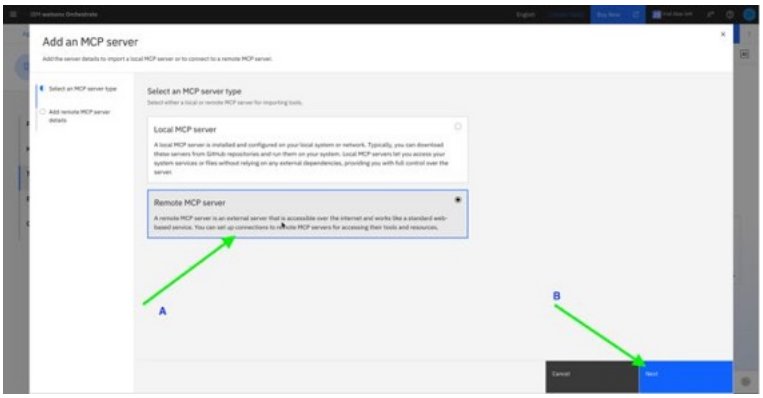
14. Select "MCP server" from the popup modal:



15. Click on **Add MCP server**



16. Next, select **Remote MCP server** and click **Next**



17. Enter the following information:

Server name: *Pipeworx_Holiday_MCPserver_<your_initials>*

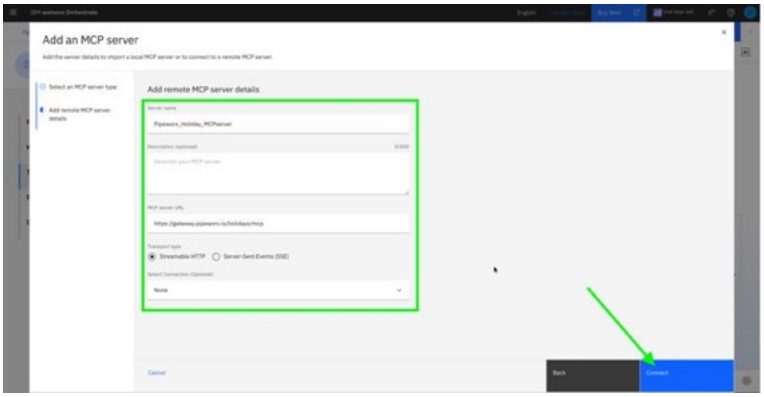
⚠️ make sure you pick a unique name – All MCP servers across the IBM Cloud need to have unique IDs (like domain names)

MCP server URL: https://gateway.pipeworx.io/holidays/mcp

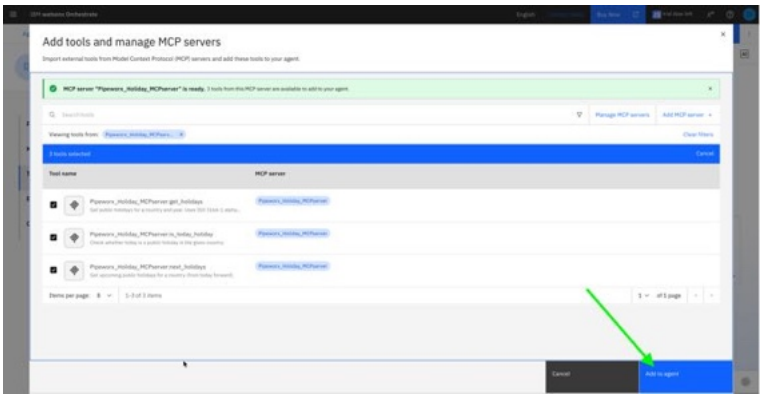
Transport Type: *Streamable HTTP*

Select Connection (optional): *None* (since this is a public MCP server),

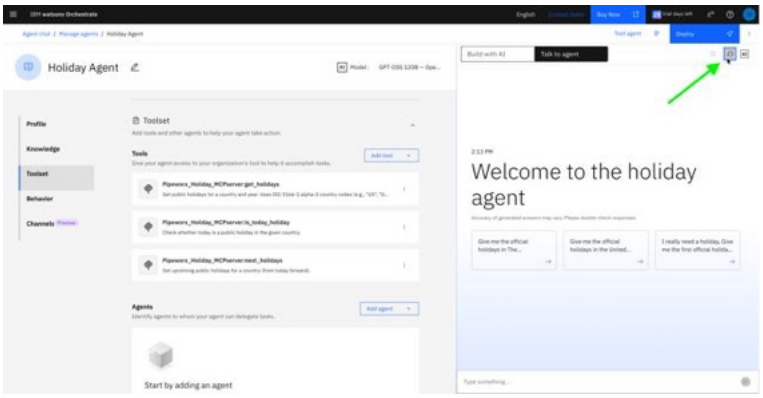
Then, click **Connect**



18. When all went well, you will receive the notification that the MCP server is ready. Now, **select all three available tools** and click **Add to agent.** The three tools will be added to the agent.

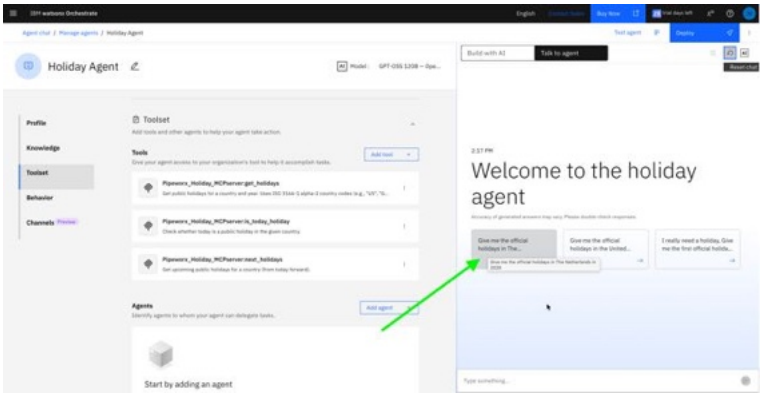


19. Now, click the **refresh button**. If everything is configured well, you should see a screen similar as below:

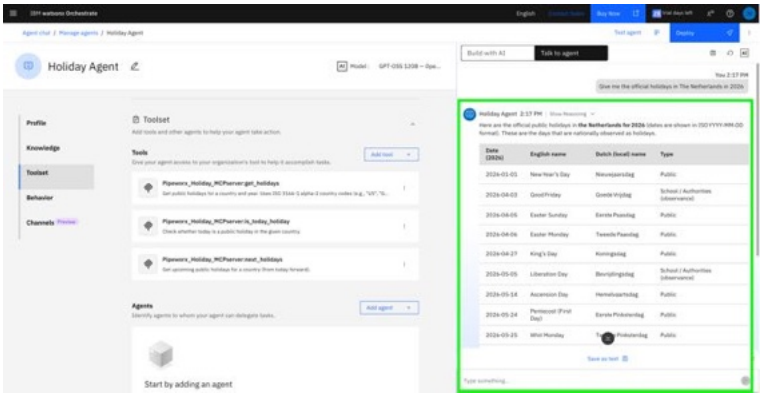


Now it is time for your first test with the Holiday Agent that collects national holidays of 100+ countries through the Pipeworx MCP server.

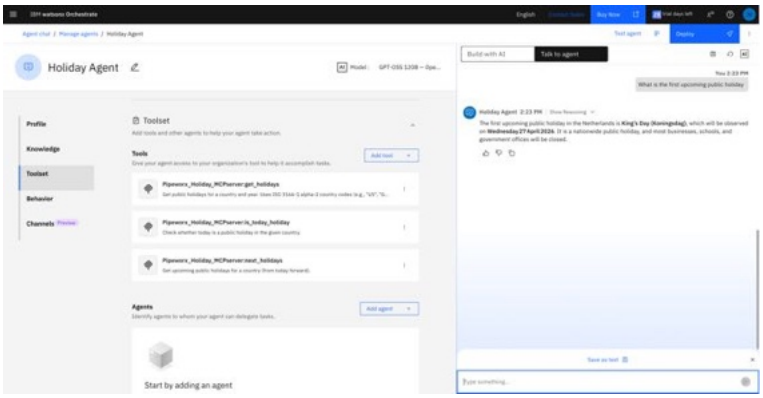
20. Click the left hand side **Quick Prompt button** in the chat window and wait for the response



21. If all went well, you should see an overview of public holidays in The Netherlands for 2026

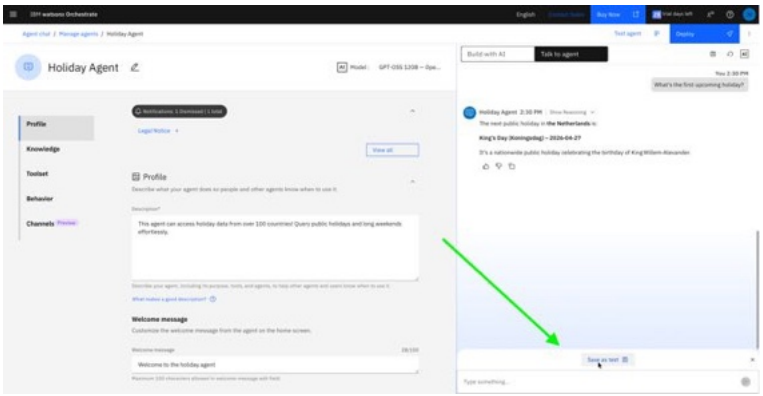


22. Next, perform another test query: *What is the first upcoming public holiday?*

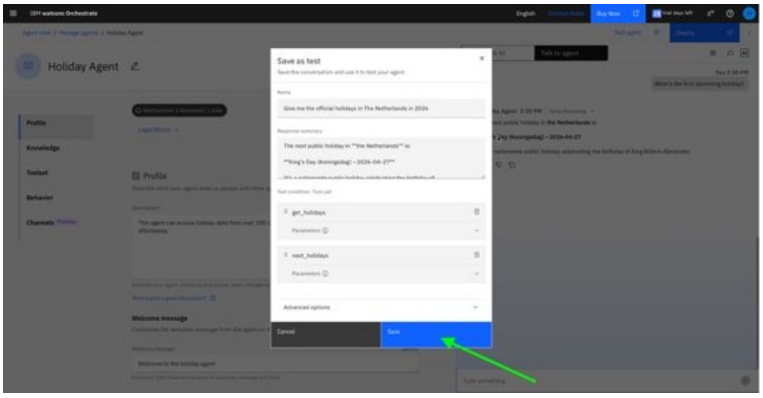


The lab manual was drafted on April 14th 2026. As you can notice from the table, the first upcoming public holiday is **King's Day** (April 27th). The agent is date context aware. Without having to specifically program or instruct it, the agent properly returns the correct date context information.

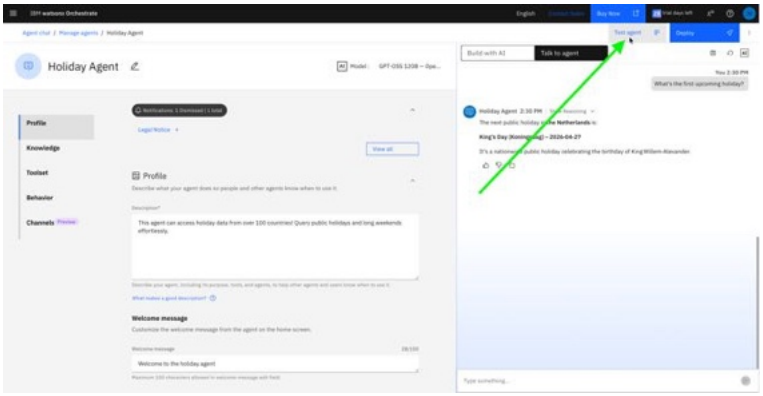
23. If you have done some test driving, it is time to test and evaluate a conversation. Restart the chat and click on of the quick prompt tiles. Wait for the response to appear. Next, save the conversation by clicking **Save as test**



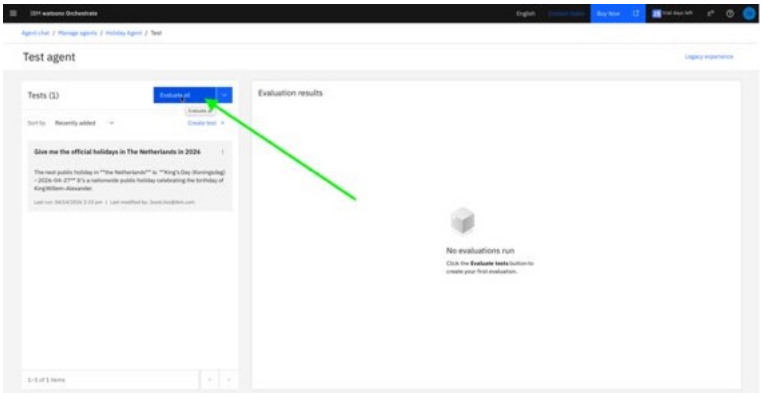
Next, click on save



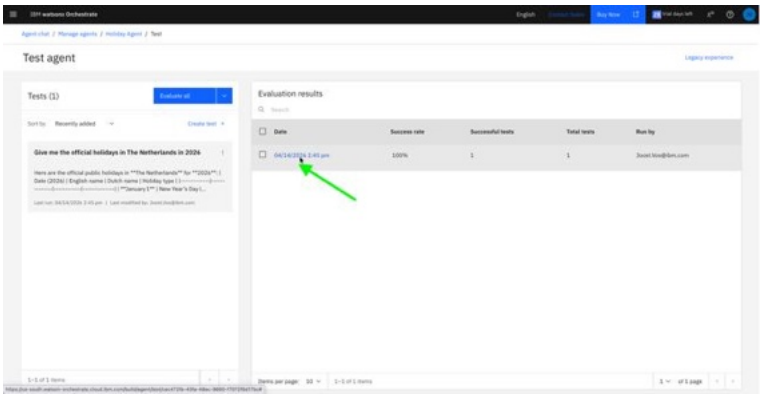
24. After saving the test, you can evaluate your agent. You can do this by clicking **Test agent**



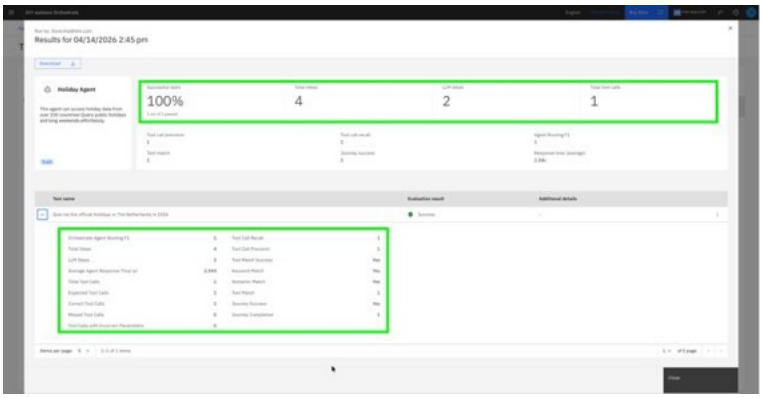
25. Next, click on the blue **Evaluate all** button to start testing your agent



26. After the evaluation has complete, click on the **Evaluation result** you want to inspect



27. A popup model opens that displays the **Evaluation results** in detail, as shown below

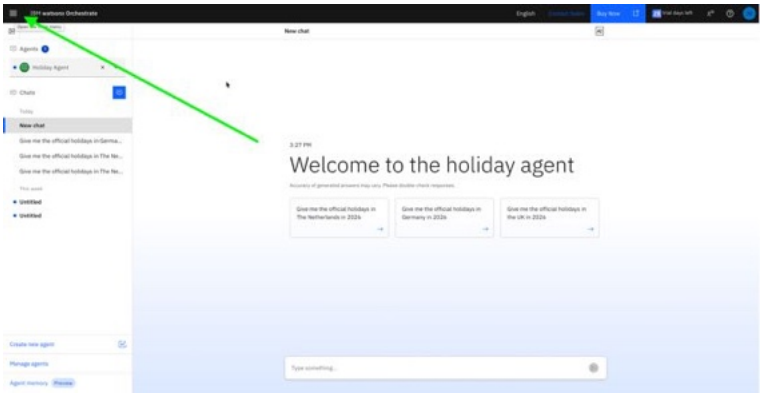


🎉 Congratulations! You have finished the first part of this lab!

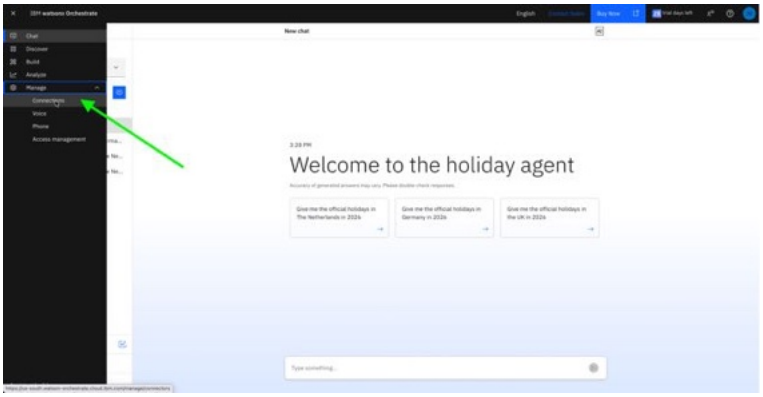
3.2 Part 2 - Adding a remote MCP server with authentication

In this exercise, you will using a remote MCP server that uses authentication credentials. Before adding a (remote) MCP server that requires authentication, you first need to create a **Connection** and add **Credentials**. As preparation you created an account with Apify and you collected the personal API key. You will need this for creating the **Connection**. Let's go!

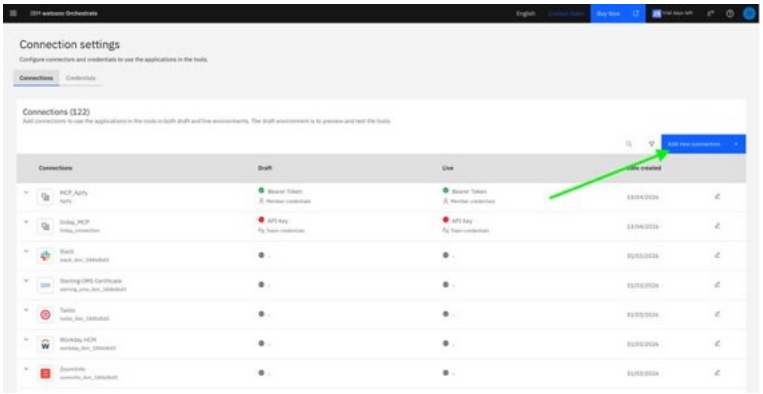
1. Open the **hamburger menu** in the top left of the navigation bar



2. Next, click **Manage** → **Connections**



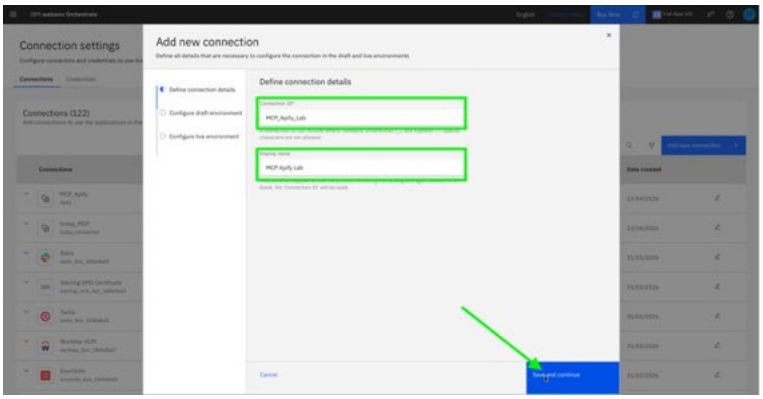
In the next screen, click on the blue **Add new connection** button



3. A popup modal opens. Enter the following information:

   Connection ID: MCP_Apify_Lab_<your_initials>

   Display name: MCP Apify Lab <your initials>



   And click **Save and continue**

4. Add the new connection. Enter the following information:

   Authentication Type: Bearer Token

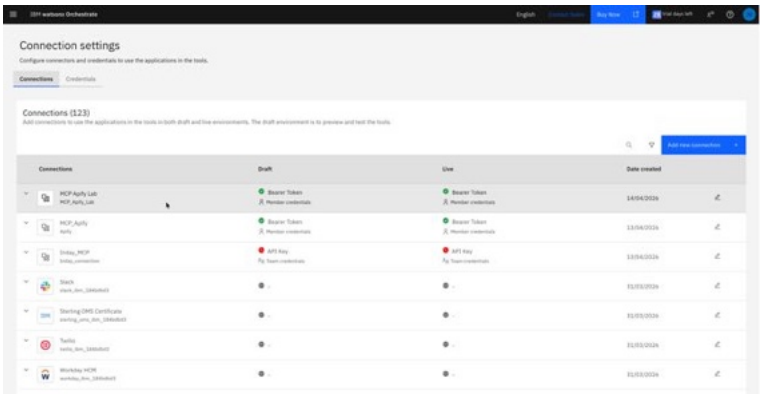
   Server URL:

   https://mcp.apify.com/?tools=call-actor,easyapi/public-holidays-api

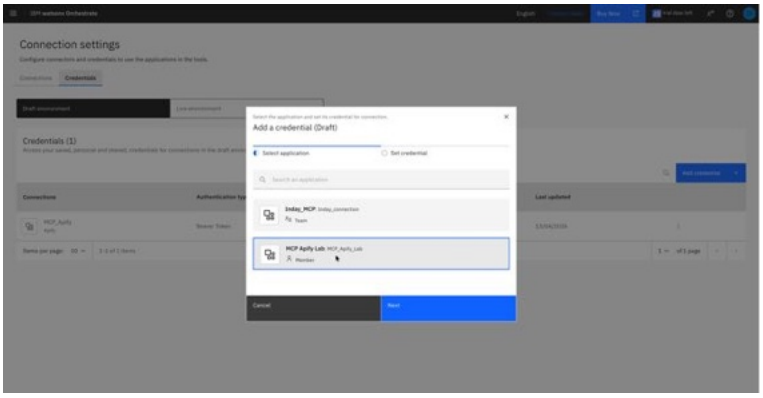
   Select Member credentials radio button

5. For configuring the live environment, it suffices to click on the **Past draft configuration**-button. Finally, click on the blue **Finish**-button

6. Now you are done with configuring the remote MCP server with authentication. If all details were submitted correctly, you should see an image similar as below:

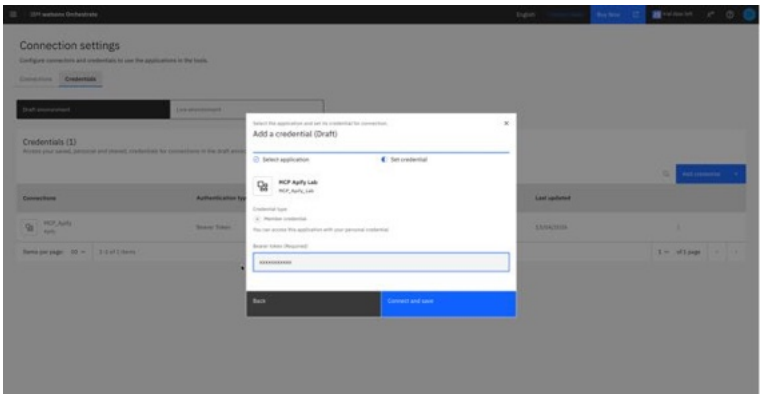


7. Next, navigate to **Credentials → Add credentials**

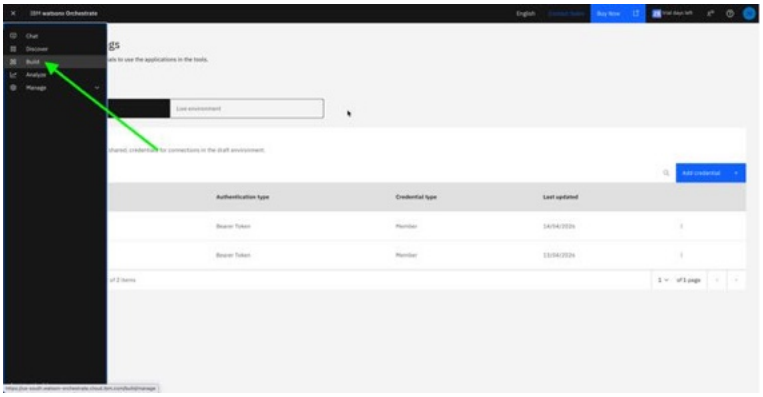


8. Select the **MCP Apify Lab <your intials>** and click **Next**

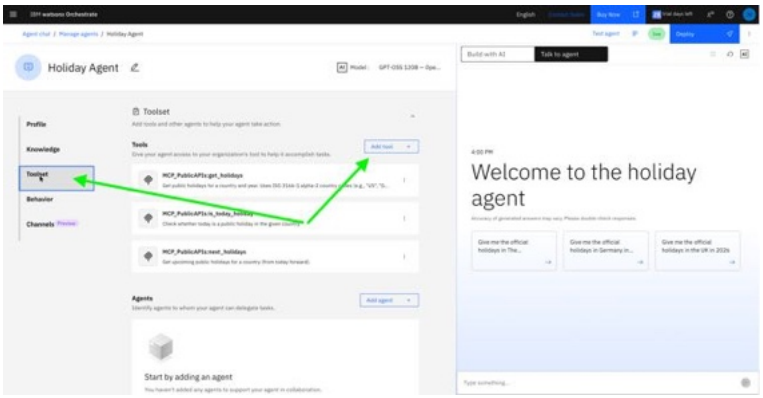
9. Next, enter the Bearer token, this is the API key from Apify (see p7), and click on **Connect and save.**



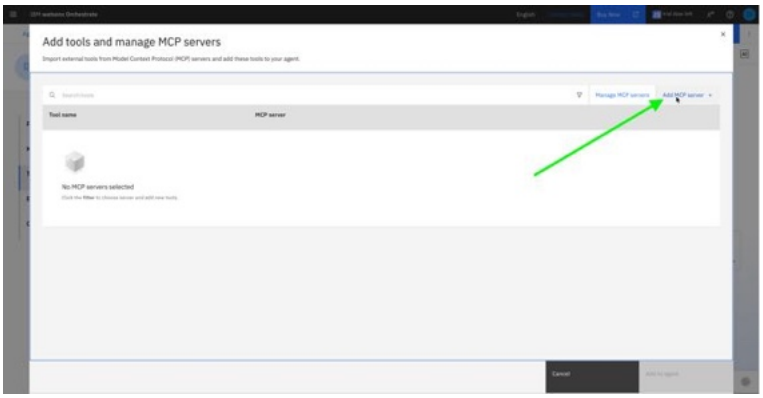
10. Now, head over to your holiday agent by clicking the **hamburger menu** in the **navigation bar → Build →** click on **the pane of your holiday agent**



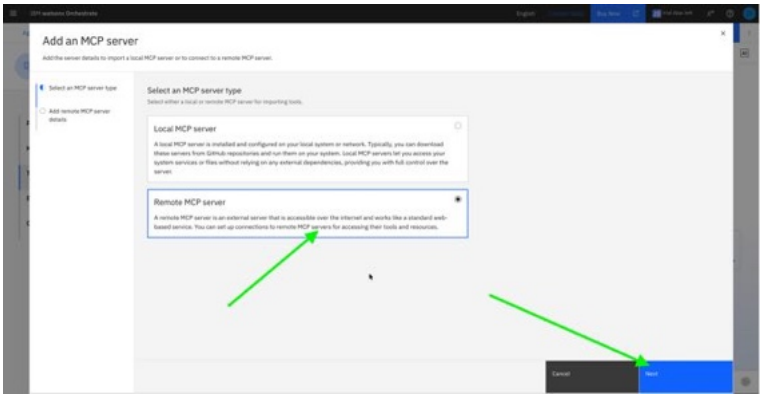
11. Next, click on Toolset in the left side menu and next click on **Add tool**



12. In the popup modal, click on **Add MCP server**



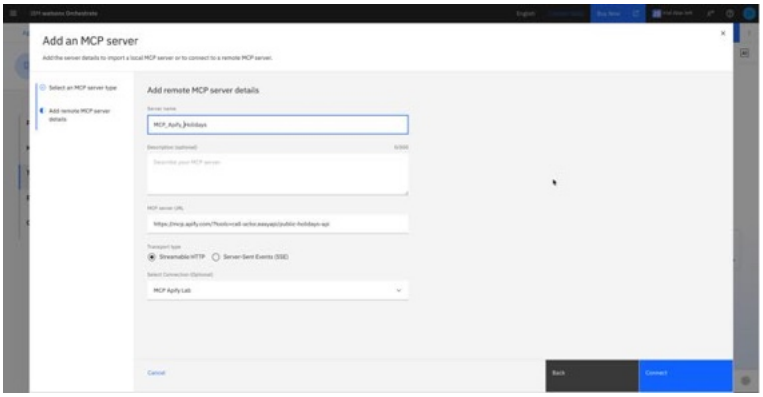
13.Select the Remote MCP server option and click **Next**



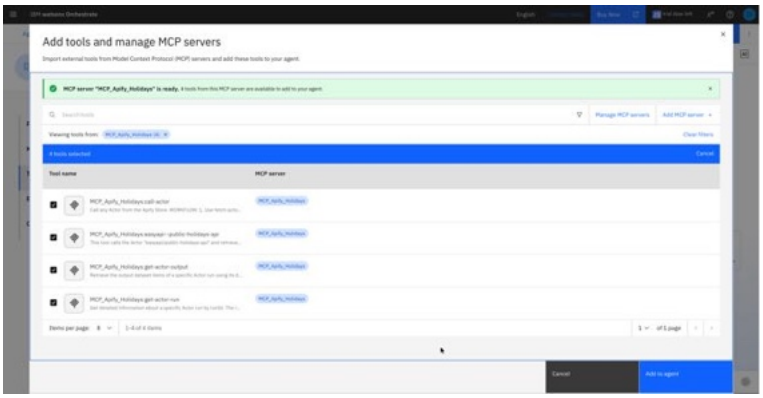
14. Enter the following information

   a. **Server name:** MCP_Apify_Holidays_<your_initials>
   b. **MCP server URL:** https://mcp.apify.com/?tools=call-actor,easyapi/public-holidays-api
   c. **Transport type:** Streamable HTTP
   d. **Select Connection:** MCP Apify Lab (from dropdown menu)

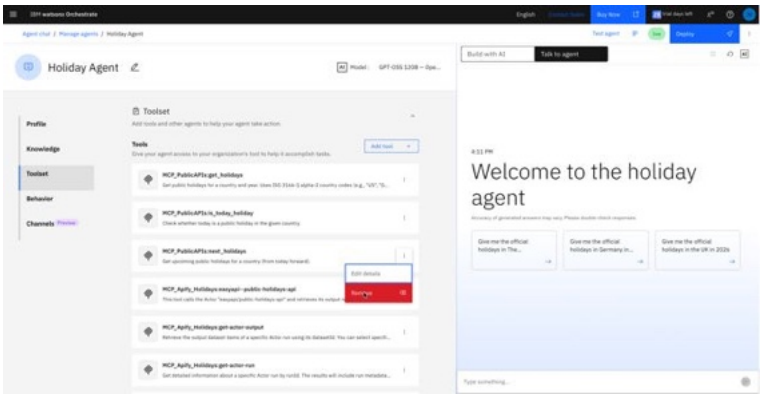
Next, click **Connect**



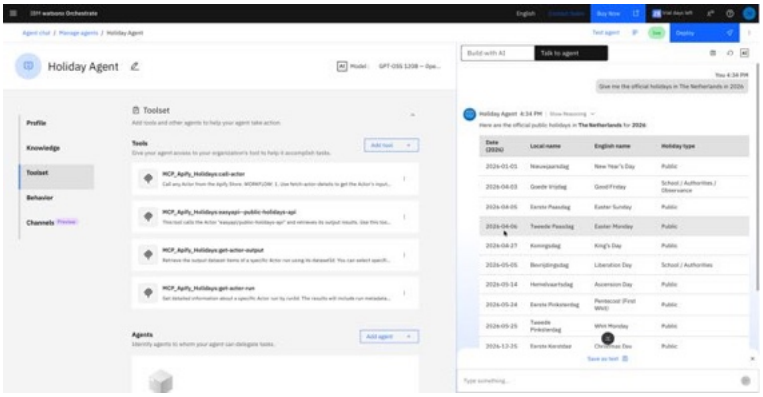
15.Select all the tools and click **Add to agent**



16. Next, remove the public MCP server by manually removing the tools. To do so, click on the three vertically aligned dots on the right-hand side of the tool lines.



17. You now have replaced the one MCP server for another. Let's do a test drive! Reload the chat and press one of the Quick start prompt tiles. If things worked out correctly, you should be able to reproduce the image below:



🎉 Congratulations! You have finished the second part of this lab!

In this part of the lab, you have seen how adding remote MCP servers that require authentication work in IBM watsonx Orchestrate. Next to that, you even have replaced one MCP server for another! While test driving, as a user, haven't probably even noticed the seamless change of backend systems. Have you experienced any obstacles in retrieving the Holiday output? What you've accomplished is basically changing one backend system for another, without having the user to bother with organizational IT changes. Imagine such IT flexibility! Users remain working in a unified user interface, while you can swap applications in the backend. By using MCP servers, managing and maintaining APIs as tools to be consumed in AI applications becomes much more standardized, thereby easing its maintenance, control and governance.

4 Multiagent systems and IBM watsonx Orchestrate

A single AI agent can be impressive on its own, but real business problems rarely live inside a single system or a single domain. Onboarding an employee touches HR, IT, facilities, and finance. Resolving a supply-chain disruption involves procurement, logistics, and planning. No single agent can be an expert in all those areas at once—and even if it could, cramming everything into one monolithic agent makes it fragile, hard to test, and difficult to govern. That's where **multi-agent systems** come in: instead of one agent doing everything, you build **specialized agents** (an HR agent, an IT agent, a finance agent) and let them collaborate—each handling what it's best at, coordinated by an orchestration layer that routes requests, manages dependencies, and ensures the overall outcome is delivered.

IBM watsonx Orchestrate is built for exactly this pattern. It acts as a **supervisor, router, and planner** across ***your agent landscape***. It decides which agent (or combination of agents) should handle a given request, passes the right context between them, and tracks progress toward the overall goal. The value is clear: *you protect existing investments (each team can build and maintain its own agent), you scale automation across business functions without creating a tangled monolith, and you retain central governance over what agents can do and when humans need to approve*.

4.1 The Agent2Agent (A2A) protocol

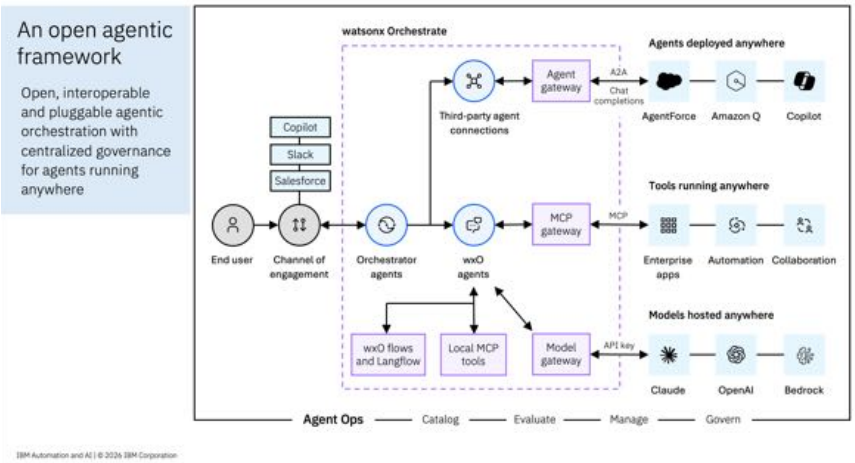
The obvious question in a multi-agent world is: *how do agents from different teams, vendors, or frameworks talk to each other?* Exactly this question is addressed by the **Agent2Agent (A2A)** protocol. A2A is an open standard initially introduced by Google in April 2025. IBM was one of the founding supporters of A2A and has actively shaped its development. The protocol has since grown to over 150 supporting organizations—including AWS, Cisco, Microsoft, and many others—and is now hosted under the Linux Foundation. IBM watsonx Orchestrate supports multiple A2A protocol versions. Each partner A2A agent identifies its protocol version and automatically routes communications through the correct protocol adapter. The currently supported version is 0.3.0. Be aware that earlier versions, such as 0.2.1, are deprecated and will be removed in a future release. To ensure continued support, use version 0.3.0 or later when registering A2A-compatible agents. Navigate to https://a2a-protocol.org/latest/ for more information on the **Agent2Agent protocol**.

4.1.1 Why Agent2Agent and MCP matters?

Without a standard like A2A, connecting two agents built by different teams is like connecting two phone systems that speak different languages: you'd need custom translation code for every pair. A2A eliminates that by giving all agents a **shared and standardized language** for collaboration. Combined with MCP (which standardizes how agents use tools), A2A completes the picture: **MCP connects agents to tools**, **A2A connects agents to each other**. In the context of this hands-on lab, watsonx Orchestrate sits at the centre of that architecture—coordinating specialized agents via A2A, connecting them to enterprise tools via MCP, and providing the governance and oversight layer that makes the whole system trustworthy and production-ready.

4.1.2 The IBM watsonx Orchestrate's Open Framework Architecture

IBM watsonx Orchestrate is **an agentic AI platform that embraces open standards**. *Figure 1* shows exactly why we call this an open agentic framework—and how ***watsonx Orchestrate*** delivers interoperability with centralized governance*page 37 for agents running anywhere.



IBM's state-of-the-art orchestration layer is embedded at the centre of the platform. Figuring out the goal behind the user question is one the things that separates a useful AI agent/assistant from a frustrating one. Figuring out *what users actually want* is where the real intelligence lives. That's why IBM heavily invested in the conversational layer—collecting intent, asking follow-ups, confirming before acting. The orchestration and tool-calling parts are *relatively mechanical* once you know what the user wants. Delegating user queries to third-party agent are relayed through Agent gateway that supports A2A. Tools running ***anywhere*** can be leveraged through the MCP gateway, federated across enterprise apps, automation platforms and collaboration tools. Models hosted anywhere can be consumed through Model

gateway accessible via standard API key integration. Native orchestrate agents and flows - including the open source-based Langflow integration - add another layer to ease of build. Local MCP tools provide specialized capabilities, all working seamlessly across enterprise channels and deployment options. This is the open, pluggable orchestration plane that allows enterprises to build multi-agent systems without vendor lock-in - coordinating work across the tools, data and systems they already have or will have in the future. With watsonx Orchestrate, IBM differentiates on enterprise trust, governance, and orchestration and its open framework architecture.

4.2 Let's play with A2A and create your first multiagent system.

During the morning labs – you created the document processor agent. This agent allows for classifying invoices and contracts. Once classified into the type of document, the following document extractor extracts fields of interest per document type. In the afternoon labs – you created a holiday agent. This agent allows to query for official holidays in 100+ countries. Let's combine these agents in a multiagent system!

4.2.1 The basics: How watsonx Orchestrate handles intent and routing

watsonx Orchestrate doesn't rely on a classic "train 50 intents with sample utterances" model. Instead, it uses a **supervisor agent pattern**: a central orchestrator that acts as a **supervisor, router, and planner** across your entire agent landscape. This agent is called **AskOrchestrate** *(and can be renamed if desired)*. The routing mechanism is **LLM-driven and description-based** rather than rules-based. Each agent you build has a **description** and **instructions.** Now **pay attention here!** ***The supervisor agent relies on those descriptions to decide which collaborator – or subagent - should handle a given request***. So instead of manually mapping *"if the user says X, route to agent Y"* the LLM reasons about the user's request and matches it to the agent whose description best fits. So, the description doesn't influence the behaviour of the agent itself, but it ***does*** influence whether the supervisor agent is able to discover and call the subagent.

4.2.2 Why this is interesting?

This routing approach based on **descriptions** and **instructions** has some practical implications worth understanding:

No intent training required: You don't need to curate hundreds of sample utterances per intent. **You write clear agent descriptions, and the LLM handles the matching**. This dramatically reduces setup time—especially when you have many agents. *Noteworthy, be aware of model replacements – you might need to re-optimize your description to work well with a different model!*

Dynamic routing: Because the supervisor uses LLM reasoning rather than a static intent tree, it can handle ambiguous or multi-part requests more flexibly. If a request spans two domains (e.g., "onboard this contractor and set up their project access"), the supervisor can plan a sequence across multiple agents rather than forcing the user to ask twice.

The quality of your descriptions becomes critical: This is the flip side: if your agent descriptions are vague, overlapping, or poorly written, the supervisor will make bad routing decisions. In a traditional intent engine, you fix misclassification by adding training utterances. In Orchestrate, you fix it by **writing better descriptions and instructions**. This is a different skill for the AI engineer to work on!

Orchestrator agents eliminate coordination complexity: Rather than building custom glue between systems, APIs, and user intents, the orchestrator handles the coordination between collaborators *(or subagents)* centrally.

4.2.3 The bigger A2A (and MCP) picture

Think of multiagent systems as two-layer systems:

1. **Supervisor layer:** try to grasp and understand the user's goal, plan the steps, pick the right agent(s), manage handoffs and sequencing
2. **Agent layer:** each specialized agent (collaborator or subagent) handles its domain (HR, IT, finance, etc.) and uses its own tools via MCP, and can collaborate with other agents via A2A

The supervisor doesn't just classify an intent and forget about it. Instead, it **tracks the conversation**, breaks complex requests into subtasks, and plans and coordinates the results back into a coherent response. That's what makes it orchestration rather than just routing.

5 Part 3 – Build your first multiagent system

In this exercise, you will build a multiagent system from the agents you've created earlier. When you are building and testing agents in watsonx Orchestrate, these are the key things to get right:

Agent descriptions: make them specific, non-overlapping, and clear about what the agent **does** and **doesn't** handle

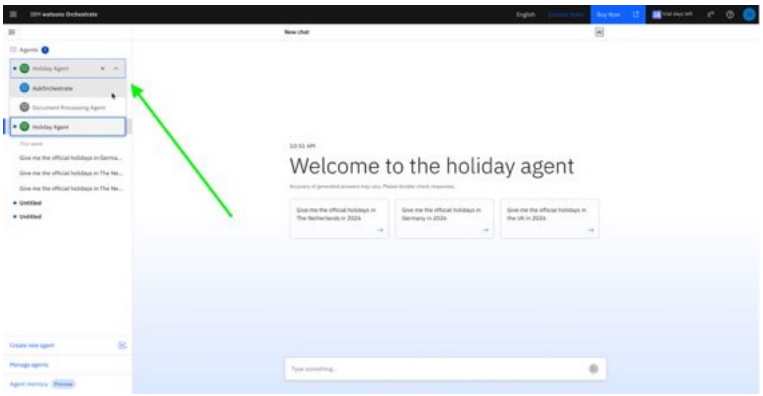
Instructions: guide the LLM's reasoning so the agent uses its tools correctly and asks the right follow-up questions

Testing with ambiguous prompts: see how the supervisor routes edge cases—this is where you learn whether your descriptions are sharp enough

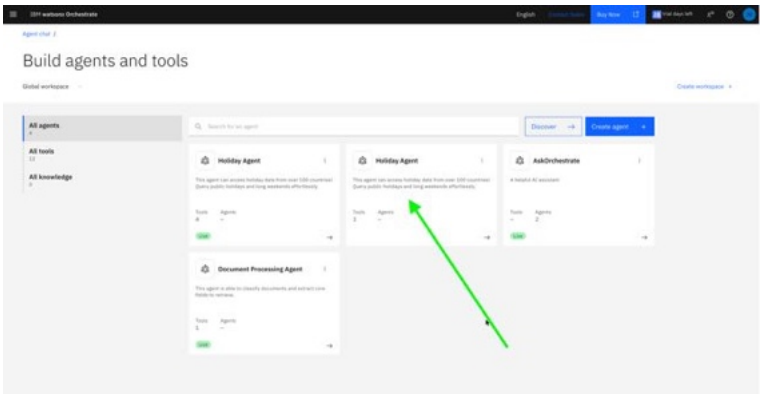
Multi-agent flows: try requests that require more than one agent and observe how the supervisor sequences the work

Let's go!

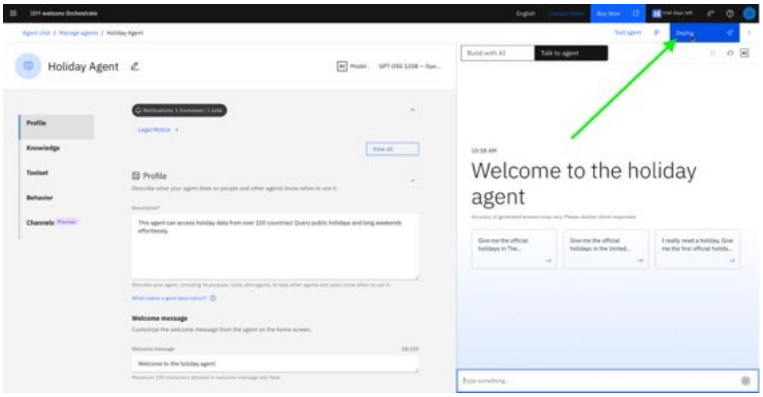
1. Click on **IBM watson Orchestrate** in the top left of the navigation bar to go to the chat screen of the product
2. Click on the **Agents flopdown menu** and make sure that your created agents are listed here.



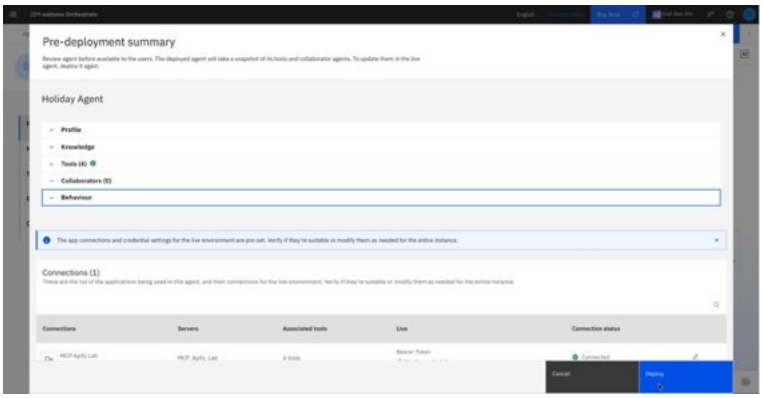
3. If you agents are not listed here, you first need to deploy them. Deploying agents is straightforward. Go to **hamburger menu** and next click **Build.** Then **click on the agent tile** you want to deploy



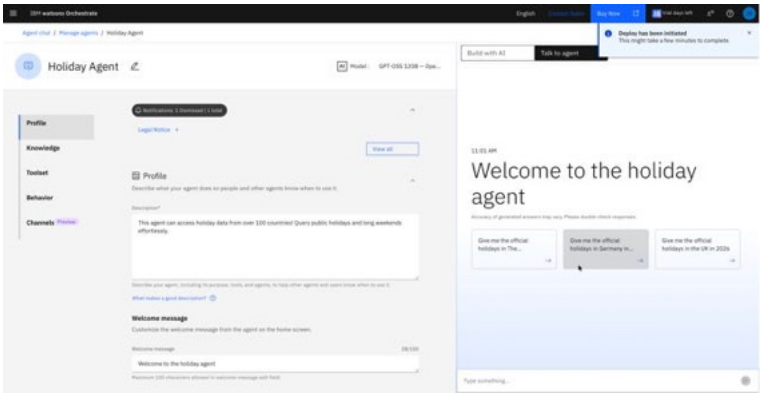
4. Hit the blue **Deploy** button



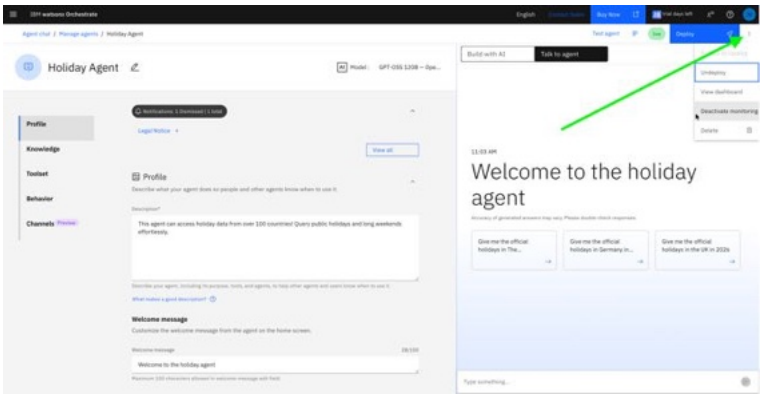
5. Check the agents' properties (Profile, Tools and Behaviour) and click **Deploy**:



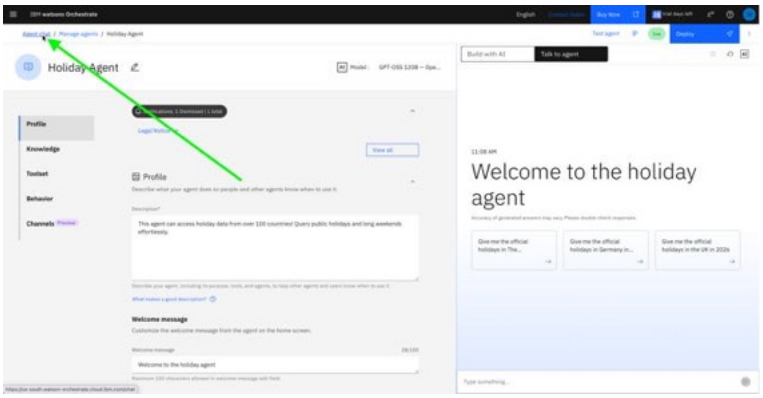
6. Deployment will be initiated. If Orchestrate asks to monitor the agent, click on **Activate agent monitoring**



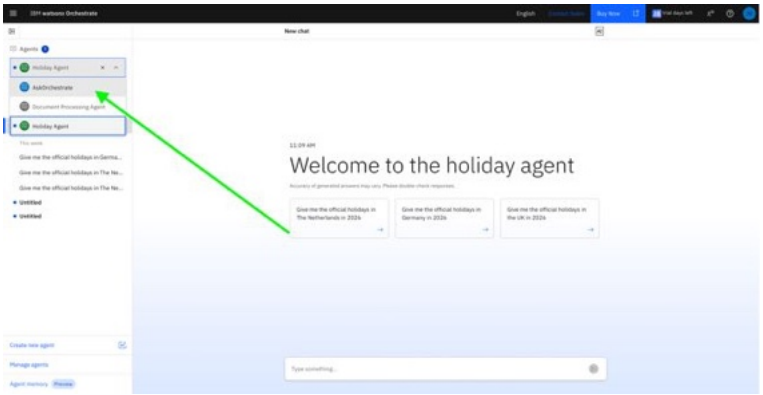
7. Optional: You can check whether agent monitoring has been activated by clicking the three dots next the **deploy**-button. There you have the option to activate/deactive monitoring



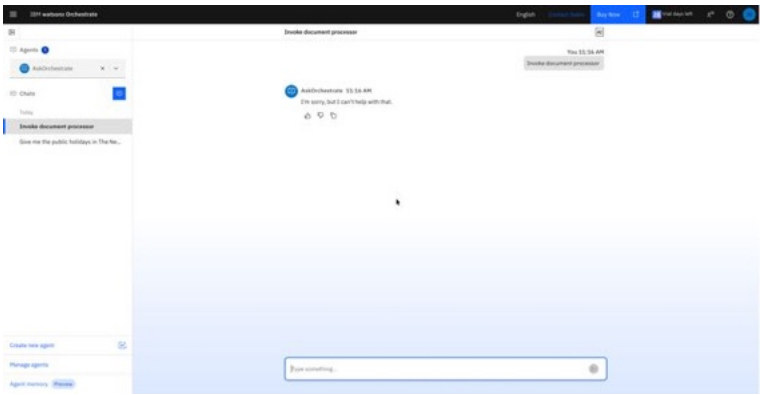
8. Please repeat step 3-7 if you haven't deployed the Document Processing agent. Always make sure that you fill out a description and a behaviour!
9. Now it is time to evaluate the router agent AskOrchestrate! Click on **Agent chat**



10. And select **AskOrchestrate**

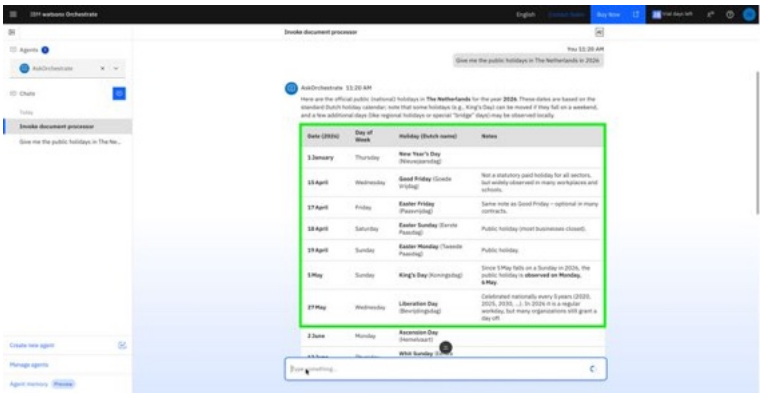


11. Ask the router AskOrchestrate agent the question:*"Invoke document processor"*



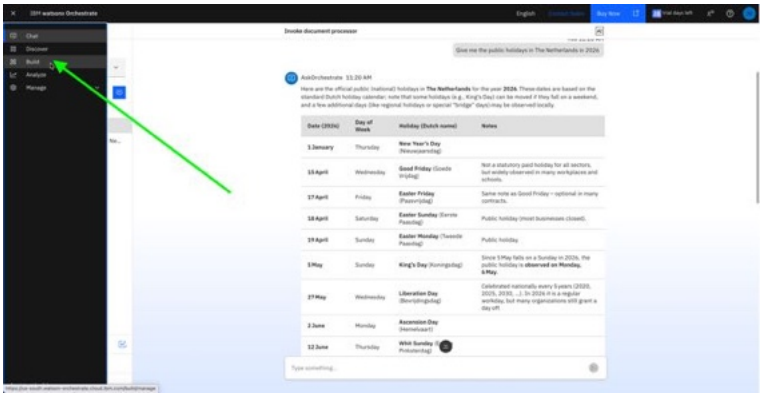
As you will notice, we didn't configure anything in the AskOrchestrate router agent. So, how can AskOrchestrate know to route this request to the Document processing agent?

12. Now ask another question:"Give me the public holidays in The Netherlands in 2026"

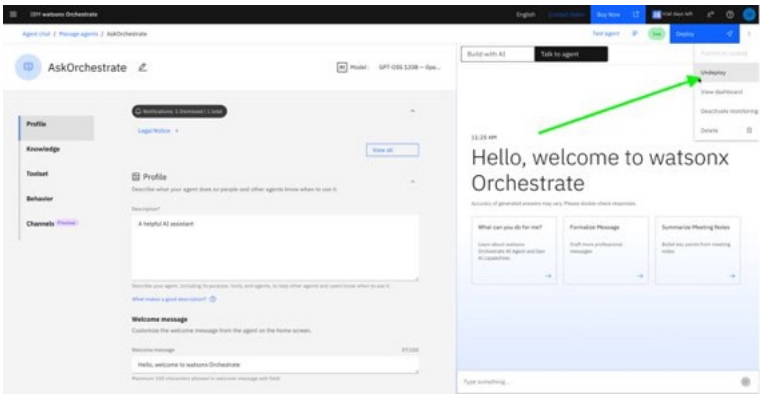


Now, look carefully at the output. Although it looks convincing, it contains major hallucinations. The model fabricated the holidays (Good Friday on a Wednesday, Kings' day on Liberation day).

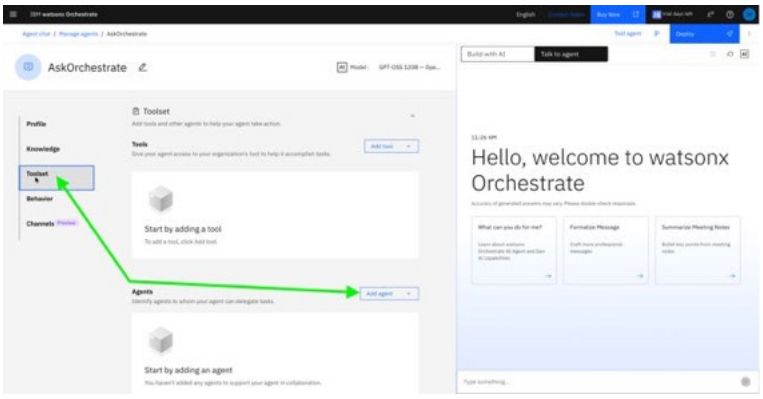
13. Now, let's build in some good delegations to the dedicated agents you created! Go to **Build** and **click on the AskOrchestrate tile**



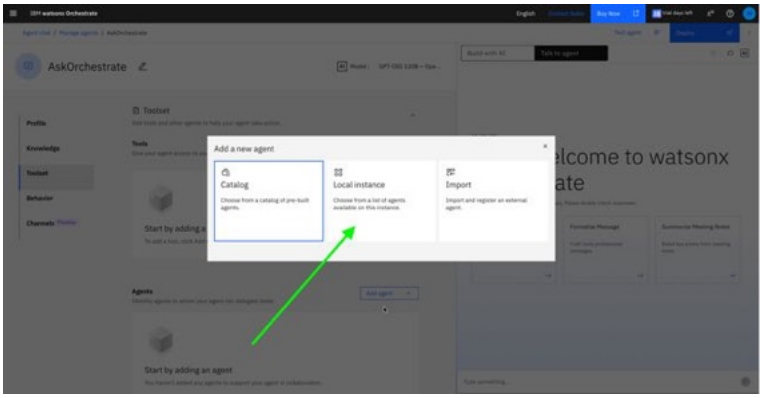
14. Next, click on the **three dots** next to the Deploy-button and click **Undeploy**.



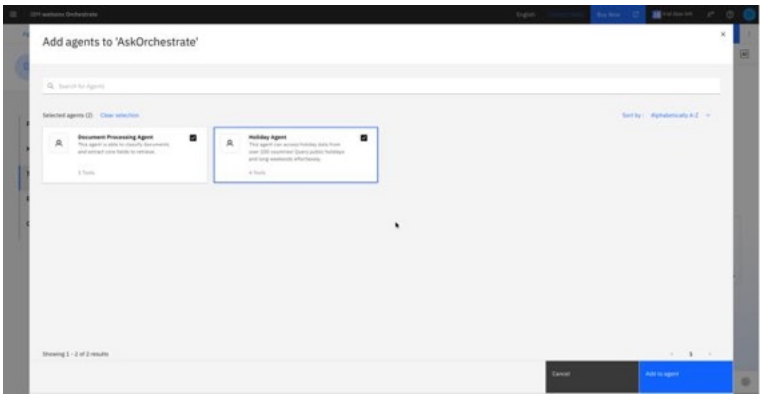
15. Next, click on **Toolset**, and next on **Add agent**



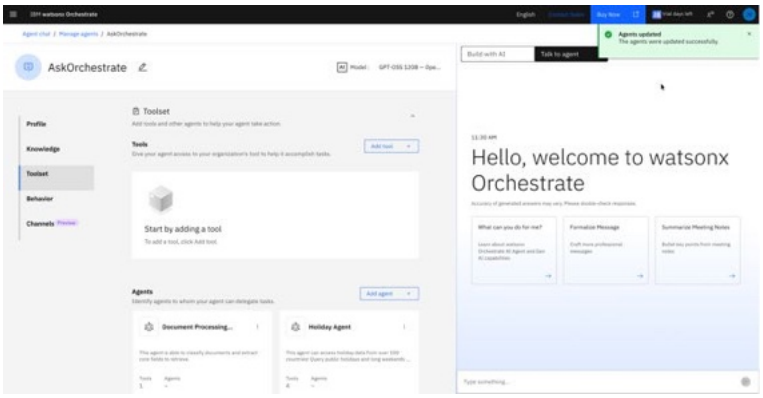
16. A popup model will appear. Click on **Local instance**



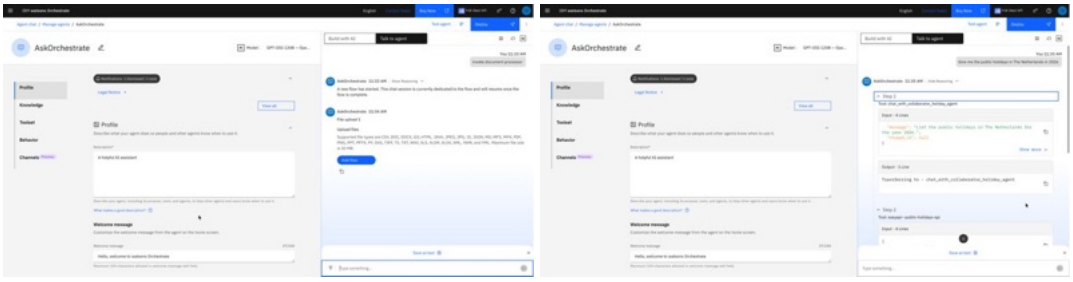
17. **Select both agents** and click on the **Add to agent**-button in the lower right corner



18. Now, you have successfully added the Document processing agent and the Holiday agent to AskOrchestrate router agent.

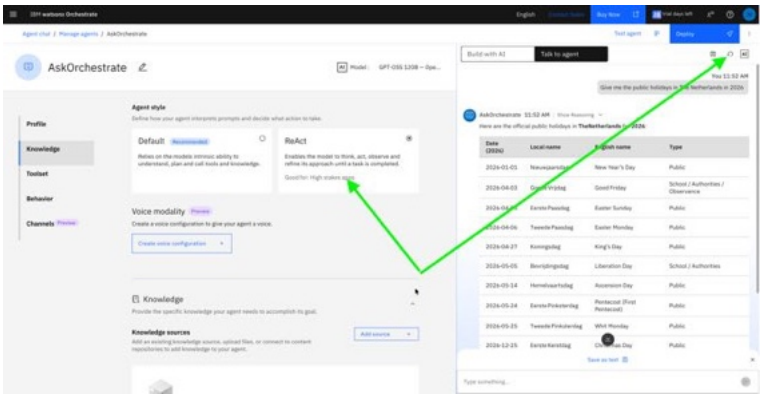


19. Let's test by asking the following two questions:
    a. Invoke document processor
    b. Give me the public holidays in The Netherlands in 2026



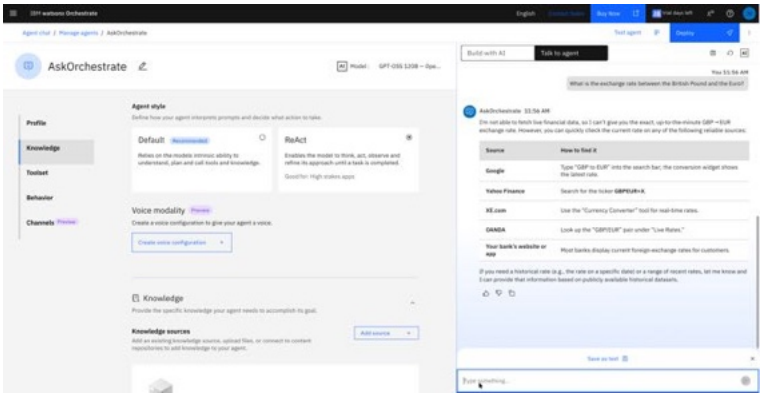
As you can appreciate, for both questions, AskOrchestrate delegates the user queries to the appropiate subagent. Now, it doesn't all look well. You may have noticed that the formatting the publical holidays is just plain text and not eligble for friendly reading. Let's tweak the multiagent system! Now you are going to optimize the instructions and behaviour!

20. First, you will modify the **Agent style.** The **agent style** defines how your agent interprets prompts and decides what action to take. Switch to **ReAct**. This will help you with the appropriate table formatting of the holidays. Next, click on reload the chat window and test the "Give me the public holidays in The Netherlands in 2026" again.



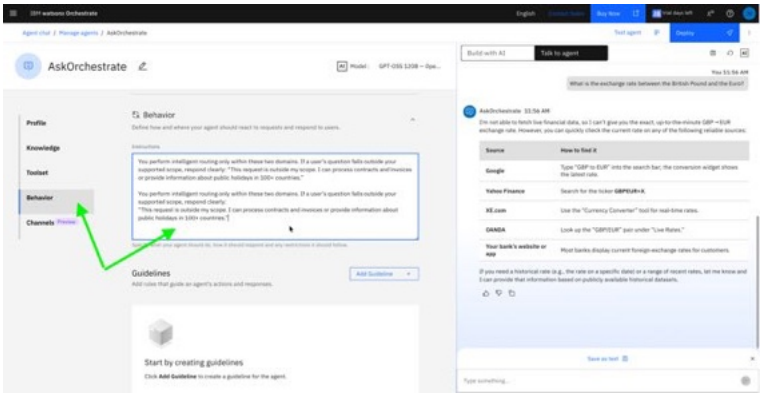
As you can appreciate, the formatting issue has been fixed by setting the agent style on ReAct.

21. Next, ask a question outside the scope of the two subagents: “What is the exchange rate between the British Pound and the Euro?”

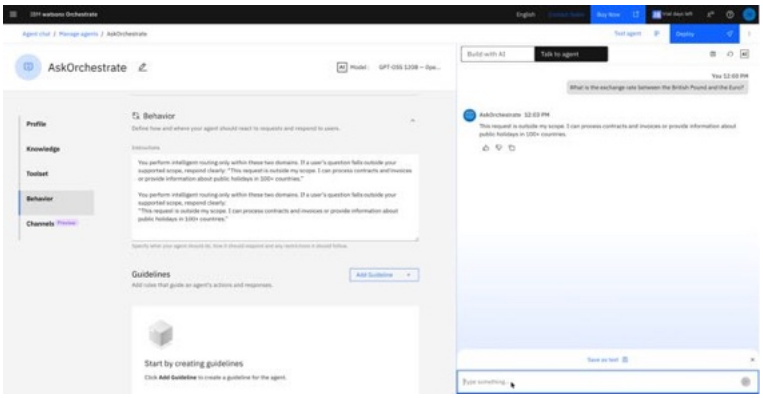


You will be getting an answer. And this is not what you want. Within enterprise settings, you would ideally confine the mutliagent its behaviour to a predetermined set of tasks you can control, maintain and govern. To minimize unwanted behaviour, we can add instructions to **Behaviour** as well as **guidelines.**

22. Next, click on **Behavior** in the left-hand side menu and add the following **instruction:**”*You perform intelligent routing only within these two domains. If a user’s question falls outside your supported scope, respond clearly: “This request is outside my scope. I can process contracts and invoices or provide information about public holidays in 100+ countries.”*
You perform intelligent routing only within these two domains. If a user’s question falls outside your supported scope, respond clearly:
“This request is outside my scope. I can process contracts and invoices or provide information about public holidays in 100+ countries.”

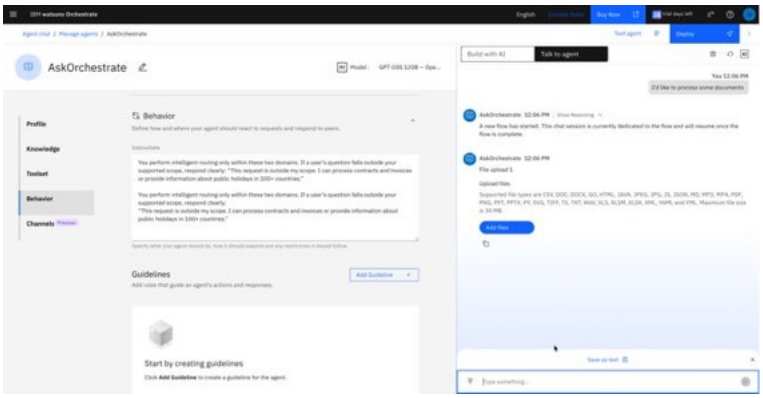


23. Restart the chat and ask the exchange rate question again:" *What is the exchange rate between the British Pound and the Euro?*" and see what happens:



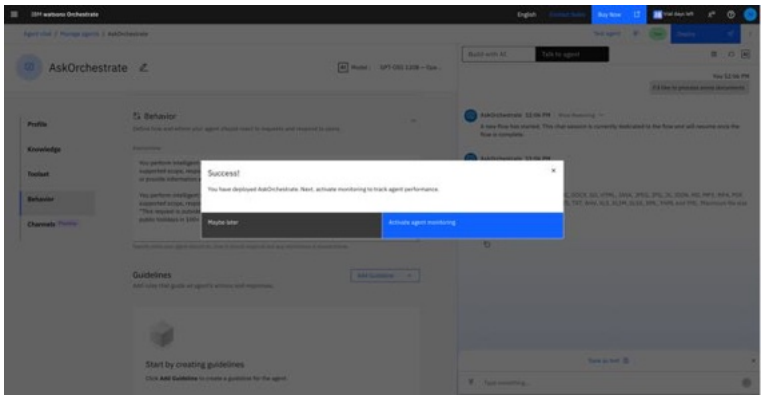
As you can see, you have been able to limit the scope of the multiagent system just to the functionality provided by the added subagents.

24. Next, test AskOrchestrate's capabilities to route to the Document processing agent by asking the following question:"I'd like to process some documents"

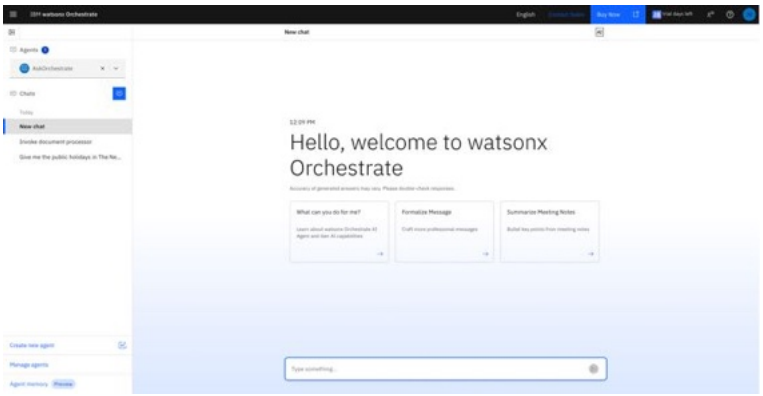


If all worked out correctly, you should have obtained a similar example as above.

25. You are ready to deploy your AskOrchestrate router agent! Hit the Deploy-button and make sure to **Activate agent monitoring**



26. Finally, go to **Chat** to see your multiagent system in real life! With that, you have successfully completed this lab! You are ready to rumble!



🎉 Congratulations! You have successfully completed this lab! 🎉

➡️You have switched on **Agent monitoring**. If there's time left, your instructors will live demo what you can do with Agent monitoring and Agent traceability.